

- *module.o* is a filename target that depends on *module.c* and *module.h*; it calls GCC to compile the *module.c* file to produce *module.o*.
- *clean* is a special target that has no dependencies, but specifies the commands to clean the compilation outputs from the project directories.

You may be wondering why the order of the *make* targets and commands in the *Makefile* are not the same as that of the manual compilation commands we ran earlier. The reason is so that the easiest invocation, by just calling the *make* command, will result in the most commonly desired output—the final executable. How does this work? The *make* command accepts a *target* parameter (one of those defined in the *Makefile*), so the generic command line syntax is *make <target>*. However, *make* also works if you do not specify any *target* on the command line, saving you a little typing; in such a case, it defaults to the **first** target defined in the *Makefile*. In our *Makefile*, that is the target *all*, which results in the creation of the desired executable binary *target_bin*!

Makefile processing, in general

When the *make* command is executed, it looks for a file named '*makefile*' or '*Makefile*' in the current directory. It parses the found *Makefile*, and constructs a dependency tree. Based on the desired *make* target specified (or implied) on the command-line, *make* checks if the dependency files of that target exist. And (for filename targets—explained below) if they exist, whether they are newer than the target itself, by comparing file timestamps. Before executing the action (commands) corresponding to the desired target, its dependencies must be met; when they are not met, the targets corresponding to the unmet dependencies are executed before the given *make* target, to supply the missing dependencies.

When a target is a filename, *make* compares the timestamps of the target file and its dependency files. If the dependency filename is another target in the *Makefile*, *make* then checks the timestamps of that target's dependencies. It thus winds up recursively checking all the way down the dependency tree, to the source code files, to see if any of the files in the dependency tree are newer than their target filenames. (Of course, if the dependency files don't exist, then *make* knows it must start executing the *make* targets from the 'lowest' point in the dependency tree, to create them.)

If *make* finds that files in the dependency tree are newer than their target, then all the targets in the affected branch of the tree are executed, starting from the 'lowest', to update the dependency files. When *make* finally returns from its recursive checking of the tree, it completes the final comparison for the desired *make* target. If the dependency files are newer than the target (which is usually the case), it runs the command(s) for the desired *make* target. This process is how *make* saves time, by executing only commands that need to be executed, based on which of the source files (listed as dependencies) have been updated, and

have a newer timestamp than their target.

Now, when a target is **not** a filename (like *all* and *clean* in our *Makefile*, which we called 'special targets'), *make* obviously cannot compare timestamps to check whether the target's dependencies are newer. Therefore, such a target is always executed, if specified (or implied) on the command line.

For the execution of each target, *make* prints the actions while executing them. Note that each of the actions (shell commands written on a line) are executed in a **separate sub-shell**. If an action changes the shell environment, such a change is restricted to the sub-shell for that action line only. For example, if one action line contains a command like *cd newdir*, the current directory will be changed only for that line/action; for the next line/action, the current directory will be unchanged.

Processing our Makefile

After understanding how *make* processes *Makefiles*, let's run *make* on our own *Makefile*, and see how it is processed to illustrate how it works. In the project directory, we run the following command:

```
slynn@freedom:~$ make
gcc -I . -c main.c
gcc -I . -c module.c
gcc main.o module.o -o target_bin
```

What has happened here? When we ran *make* without specifying a target on the command line, it defaulted to the first target in our *Makefile*—that is, the target *all*. This target's dependencies are *module.o* and *main.o*. Since these files do not exist on our first run of *make* for this project, *make* notes that it must execute the targets *main.o* and *module.o*. These targets, in turn, produce the *main.o* and *module.o* files by executing the corresponding actions/commands. Finally, *make* executes the command for the target *all*. Thus, we obtain our desired output, *target_bin*.

If we immediately run *make* again, without changing any of the source files, we will see that only the command for the target *all* is executed:

```
slynn@freedom:~$ make
gcc main.o module.o -o target_bin
```

Though *make* checked the dependency tree, neither of the dependency targets (*module.o* and *main.o*) had their own dependency files bearing a later timestamp than the dependency target filename. Therefore, *make* rightly did not execute the commands for the dependency targets. As we mentioned earlier, since the target *all* is not a filename, *make* cannot compare file timestamps, and thus executes the action/command for this target.

Now, we update *module.c* by adding a statement `printf("first update");` inside the *sample_func()* function. We then run *make* again: